

Informe de Pruebas Unitarias

Proyecto: Colegio O'Higgins Platform — Fullstack III (DSY1106)

Fecha: 23/06/2026

1. Resumen Global

Módulo	Tests	Estado	Cobertura (Instrucciones)
ms-students	22	100% OK	80%
ms-attendance	66	100% OK	84%
backend-bff	10	100% OK	83%
api-gateway	9	100% OK	— (sin JaCoCo)
frontend (Vitest)	349	100% OK	~28%
Total	456	100%	≥80% en backend

2. Estrategia de Pruebas

- Backend:** JUnit 5 + Mockito para pruebas unitarias de servicios, controladores y repositorios sin levantar contexto completo de base de datos.
- Frontend:** Vitest + React Testing Library para pruebas unitarias e integración de componentes.
- Patrones evaluados:** Se verificó la aplicación de Factory Pattern, Strategy Pattern, Circuit Breaker (Resilience4j) y JWT Security Filter mediante pruebas específicas.

3. Resultados por Módulo

3.1 ms-students (22 tests)

- StudentControllerTest: 8 tests (crear, listar, obtener por ID, actualizar, manejo de errores 404, RUT inválido)
- StudentServiceTest: lógica de negocio con mocking de repositorio
- GlobalExceptionHandler: manejo de excepciones HTTP

3.2 ms-attendance (66 tests)

- ServicioAsistenciaTest: lógica de registro de asistencia
- ClienteEstudiantesResilienceTest: validación de Circuit Breaker con Resilience4j (sliding window, fallback)
- ControladorAsistenciaTest: endpoints REST con validación de parámetros
- Validation Strategy Tests: PresenteValidationStrategy, AtrasoValidationStrategy, InasistenciaValidationStrategy
- Factory Tests: PresenteFactory, AtrasoFactory, InasistenciaFactory
- Model Tests: entidades JPA
- GlobalExceptionHandler: manejo de excepciones

3.3 backend-bff (10 tests)

- Health checks: verificación de estado de servicios
- Integración con RestTemplate: pruebas de comunicación con microservicios

3.4 api-gateway (9 tests)

- AuthControllerTest: login, register, refresh token
- JwtTokenProviderTest: generación y validación de tokens
- CorsConfigTest: configuración CORS

3.5 Frontend (349 tests)

- Unitarios (10 suites): authMockService, bffClient, componentes, hooks
- Integración (8 suites): Login, Register, flujo completo de autenticación
- Manejo de errores: timeouts, respuestas 401/403, excepciones de red

4. Cobertura de Código (JaCoCo)

Módulo	Cobertura Instrucciones	Cobertura Ramas	Clases Analizadas
ms-students	80%	66%	10
ms-attendance	84%	75%	20
backend-bff	83%	100%	4

Todos los módulos superan el 60% de cobertura mínima exigida por la rúbrica.

5. Cómo Ejecutar las Pruebas

Backend (Maven)

```
cd ms-students    && .\mvnw clean test
cd ms-attendance  && .\mvnw clean test
cd backend-bff    && .\mvnw clean test
cd api-gateway    && .\mvnw clean test
```

Frontend (Vitest)

```
cd frontend && npm run test:coverage
```

Reportes JaCoCo (cobertura)

```
cd ms-students    && .\mvnw clean verify
cd ms-attendance  && .\mvnw clean verify
cd backend-bff    && .\mvnw clean verify
```

Los reportes HTML se generan en `target/site/jacoco/index.html` de cada módulo.

6. Conclusión

Se implementaron **456 pruebas unitarias** distribuidas en los 5 módulos del sistema (4 backend + 1 frontend), con una tasa de aprobación del **100%** y cobertura de instrucciones superior al **80%** en los módulos backend analizados con JaCoCo, superando ampliamente el 60% mínimo exigido.